

Skriftlig prøve: 24 maj 2023

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–20%, Opgave 2–30%, Opgave 3–30%, Opgave 4–20%

Note: Hvis du mangler specifikationer, kan du tilføje dine egne og fortsætte opgaveløsningen. Ikke alle begreber/beskrivelser er oversat til dansk. Eksempelvis er ordene "runtime stack" ikke oversat. I enkelte tilfælde er ikke oversatte betegnelser anført i anførselstegn ("..."). Du skal aflevere din besvarelse som et pdf-dokument med bilag, hvor du vedhæfter kildekoden til de programmer, du har skrevet i dine løsninger (filer i format c, asm, bin, etc.). Det skal være klart fra filens navn, hvilken opgave, den svarer til.

Opgave 1 (20%)

Besvar de følgende spørgsmål:

- a) Den 10 bits "2's complement"-repræsentation af det decimale tal 234 er "0011101010". Hvad er 10 bits "2's complement"-repræsentationen af den decimale tal -234?
1100010110
- b) Hvad er 8 bits "2's complement"-repræsentationen af det decimale tal -127?
10000001
- c) Vi har en ISA ("Instruction Set Architecture") hvor de øverste 6 bit i instruktionerne er brugt til at repræsentere "the opcode". Hvad er den maksimale antal instruktioner som vi kan have for dette instruktionssæt?
64
- d) John prøver at skrive et lille LC3 assembler-program som laver en addition mellem register R0 og værdien 37 og gemmer resultatet tilbage i register R0. Hans program indeholder kun instruktionen: ADD R0, R0, #37. Er det her program rigtig? Hvis nej, forklar hvad problemet er og omskriv programmet så det laver den ønskede addition mellem R0 og værdien 37.

Den maks. immediate værdi for ADD instruktionen er +15. Vi kan omskrive programmet på forskellige måde:

1)

```
ADD R0, R0, #15
ADD R0, R0, #15
ADD R0, R0, #7
```

2)

```
LD R1, A
ADD R0, R0, R1
A .FILL #37
```

Ulempen med løsning 2 er at vi bruge også register 1. Fordelen er at løsning 2 kan klare meget store værdier uden behov for ekstra instruktioner. For eksempel hvis vi skal addere R0 med 10000, kun løsning 2 vil være praktisk.

Skriftlig prøve: 24 maj 2023

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–20%, Opgave 2–30%, Opgave 3–30%, Opgave 4–20%

Opgave 2 (30%)

- a) Skriv et LC3 assembler-program, der krypterer beskeder. Den umaskerede tekst er "This is my secret message!", og den er gemt i programmet som en del af hukommelsen. Programmet beregner den maskerede tekst ved at tilføje 5 til hver ASCII-værdi fra den originale tekst (inklusiv mellemrum) og viser derefter den maskerede tekst på skærmen. Således vil de første fire bogstaver, "This", blive maskeret til "Ymnx".

```
.ORIG X3000
LEA R3, S
L   LDR R0, R3, #0
    BRz H
    ADD R0, R0, #5
    OUT
    ADD R3, R3, #1
    BRnzp L
H   HALT
S   .STRINGZ "This is my secret message!"
.END
```

- b) Hvad er den fulde krypterede tekst?

Ymnx%nx%r~%xjhwjy%rjxxflj&

Opgave 3 (30%)

Tom fik en eksamensopgave hvor han skal implementere 3 funktioner:

- `readString` – Denne funktion skal kunne læse en tekstlinje indtastet fra tastaturet og returnere både teksten og antallet af ord, som teksten indeholder. Teksten må højst indeholde 99 bogstaver. Ordene bliver adskilte af "white space" (mellemrum eller tab).
- `void firstLettersPositions(char* string, int* posArray)` – Denne funktion skal kunne identificere det første bogstav i hvert ord i teksten, som argumentet string peger på, og derefter gemme bogstavernes positioner i det integer array, som posArray peger på. Dette array er allerede allokeret i hukommelsen, før funktionen `firstLettersPositions` bliver kaldt. Arrayets størrelse er lig med antallet af ord i teksten plus en. Før funktionen `firstLettersPositions` bliver kaldt, initialiseres `posArray[0]` med antallet af ord i teksten. For eksempel, hvis string peger på teksten " One short example", vil posArray indeholde værdierne [3, 2, 6, 12]. Bemærk at der er to mellemrum før det første bogstav i teksten.
- `void showPosArray(char* string, int* posArray)` – Denne funktion skal kunne vise de individuelle ord, som teksten består af, og deres respektive startpositioner på skærmen.

Skriftlig prøve: 24 maj 2023

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–20%, Opgave 2–30%, Opgave 3–30%, Opgave 4–20%

Hvis de 3 funktioner er korrekt implementeret, og vi skriver teksten " One short example" på tastaturet, så vil vi få følgende output på skærmen:

```
Write an input string:  One short example
nrWords:3, string:  One short example
First letters in the string have the following positions:
The word One starts at string position 2
The word short starts at string position 6
The word example starts at string position 12
```

- a) Tom har kun nået at implementere den første funktion. Men desværre har programmet fejl og kører ikke som forventet. Ret fejlene i Toms program så det kører som det skal.

Her er Toms kode:

```
#include <stdio.h>
#include <stdlib.h>

//Function readString reads a string from the keyboard which can be up to 99
//characters long
//It returns two values: the string read from the keyboard and nrWords(the
//number of words the string consists of)
char* readString(int nrWords){
    char c; //Character read from the keyboard
    int i;  //Index in the array
    char wordStarted=0;
    //We can use the wordStarted variable to know when we have found a new
    //word
    //If wordStarted has the value 1 then it means that we are inside of a
    //word
    //If wordStarted has the value 0 then we are in the space between words
    char* buffer; //A buffer where we save the keys read from the keyboard
    printf("Write an input string:");
    while ((c=getchar())!='\n'){
        buffer[i++]=c;
        if ((c==' ') && (c!='\t'))
        {
            if (wordStarted) { //found the end of the word
                wordStarted = 0;
            }
        } else{
            nrWords++;
        }
    }
    buffer[i]='\0';
    return buffer;
}

int main() {
    char *myString=" Let's have a default string!";
    int nrWords=5;
    myString = readString(nrWords);
}
```

Skriftlig prøve: 24 maj 2023

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–20%, Opgave 2–30%, Opgave 3–30%, Opgave 4–20%

```
printf("nrWords:%d, string:%s\n",nrWords,myString);  
return 0;  
}
```

Hjælp til besvarelsen: Du kan godt ændre datatypen for returnværdien eller argumentet for funktionen hvis nødvendigt. Kik også på Toms kommentarer for at se, hvordan han har tænkt på at løse opgaven.

- b) Skriv koden for funktionen void firstLettersPositions(char* string, int* posArray). Du skal bruge denne funktionserklæring. Du kan ikke bruge andre argumenter, eller andre typer til argumenterne.

Hjælp til besvarelsen: Hvis du har ikke fået koden for funktionen readString til at køre ordentligt, du kan stadig løse denne opgave. Du skal bare kommentere linjen "myString = readString(nrWords);", i main funktionen. På den måde kan du, når du kalder funktionen firstLettersPositions, bruge de værdier de to variable er initialiseret til i starten af main funktionen.

- c) Skriv koden for funktionen void showPosArray (char* string, int* posArray). Du skal bruge denne funktionserklæring. Du kan ikke bruge andre argumenter, eller andre typer til argumenterne.

Koden for alle 3 opgaver er:

```
#include <stdio.h>  
#include <stdlib.h>  
  
char* readString(int* nrWords){  
    char c;  
    int i;  
    char* buffer=malloc(100*sizeof(char));  
    char wordStarted=0;  
    printf("Write an input string:");  
    *nrWords=0;  
    while ((c=getchar())!='\n'){  
        buffer[i++]=c;  
        if ((c==' ') || (c=='\t'))  
        {  
            if (wordStarted) { //found the end of the word  
                wordStarted = 0;  
            }  
        } else{  
            if (!wordStarted){  
                wordStarted=1;  
                (*nrWords)++;  
            }  
        }  
    }  
    buffer[i]='\0';  
}
```

Skriftlig prøve: 24 maj 2023

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–20%, Opgave 2–30%, Opgave 3–30%, Opgave 4–20%

```
    return buffer;
}

void firstLettersPositions(char* string, int* posArray){
    int i=0;
    int wordIndex=1;
    int c;
    char wordStarted=0;
    while ((c=string[i])!='\0'){
        if (c==' ' || c=='\t'){
            if (wordStarted) { //found the end of the word
                wordStarted = 0;
            }
        } else{
            if (!wordStarted){
                wordStarted=1;
                posArray[wordIndex++]=i;
            }
        }
        i++;
    }
}

void showPosArray(char* string, int* posArray) {
    int i = 1;
    int j;
    char c;
    //Printing the Letter Positions array
    printf("First letters in the string have the following positions:\n");
    while (i <= posArray[0]) {
        printf("The word ");
        j=posArray[i];
        while (((c=string[j++])!=' ') && (c!='\t') && (c!='\0'))
            printf("%c",c);
        printf(" starts at string position %d\n", posArray[i]);
        i++;
    }
}

int main() {
    char* myString=" Let's have a default string!";
    int nrWords=5;
    myString=readString(&nrWords);
    printf("nrWords:%d, string:%s\n",nrWords,myString);
    int* posArray=(int*)malloc((nrWords+1)*sizeof(int));
    posArray[0]=nrWords;
    firstLettersPositions(myString,posArray);
    showPosArray(myString,posArray);
}
```

Skriftlig prøve: 24 maj 2023

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–20%, Opgave 2–30%, Opgave 3–30%, Opgave 4–20%

Opgave 4 (20%)

Følgende C program ønskes kørt på en **LC3 processor**:

```
int main() {
    char myStr[]="Text";
    char* pStr="Another";
    int i=5;
    int j=7;
    int* p;
    int** t;

    struct S {
        char* s;
        char c;
        int k;
    } *st;
    t=&p;
    pStr++;
    p=&j;
    (**t)++;
    (*t)++;
    (*p)--;
    pStr=myStr+3;

    return 0;
}
```

- a) Vis indholdet af "activation record" for funktionen main lige før den sidste linje i koden (return 0;) bliver kørt. Det antages at "stack pointer" i R6 har værdien FFFF, når "runtime stack" er tom.

Hjælp til besvarelsen:

For et C-program som kører på en LC3 processor gælder:

- Integer og pointer variable er 16 bit (1 word)
- Hukommelsen er 16 bit (1 word) adresserbar
- Lokale variable placeres på "runtime"-stakken i samme rækkefølge som de erklæres
- Rækkefølgen, hvormed argumenterne til et funktionskald placeres på stakken, går fra højre mod venstre

Skriftlig prøve: 24 maj 2023

Kursus navn: Maskinnær programmering

Kursus nummer: 02322

Hjælpemidler: Alle hjælpemidler

Varighed: 4 timer

Vægtning: Opgave 1–20%, Opgave 2–30%, Opgave 3–30%, Opgave 4–20%

Address	Variable	Value	
FFF1	*st	?	<= R6
FFF2	**t	&p	
FFF3	*p	&j -> &i	
FFF4	j	7 -> 8	
FFF5	i	5 -> 4	
FFF6	*pStr	? -> &myStr[3]	
FFF7	myStr[0]	T	
FFF8	myStr[1]	e	
FFF9	myStr[2]	x	
FFFA	myStr[3]	t	
FFFB	myStr[4]	\0	<= R5
FFFC	DL		
FFFD	RA		
FFFE	RV		
FFFF			